KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science and Engineering

Campus Maintenance & Complaint Management System

Database Systems Laboratory Project Report

Student Name	Ahnaf Tajwar Sadi
Roll Number	2207104
Course	Database Systems Lab
Technology	Oracle SQL/PL/SQL · Node.js · Express · HTML/CSS/JS
Repository	https://github.com/ATSadi/dB-proj/
Date	July 2026

A database-driven campus operations platform for complaint submission, worker assignment, SLA enforcement, chronic-issue detection, and monthly reporting.

Table of Contents

1. Introduction

Campus facilities generate continuous maintenance demand across classrooms, laboratories, hostels, offices, and common areas. Manual complaint handling is slow, opaque, and difficult to audit. This project implements a complete **Campus Maintenance & Complaint Management System** that couples a normalized Oracle database with role-based web portals.

Students submit location-aware complaints with category and priority. Supervisors assign specialized workers, monitor SLA breaches and chronic issues, and generate monthly reports. Workers execute jobs and update availability. Admins manage the directory of students, workers, and locations. Core business rules—SLA deadlines, escalation, audit logging, performance scoring, and chronic-flag detection—are enforced inside the database using triggers, procedures, and functions.

2. Objectives

- Design a 3NF relational schema covering users, locations, complaints, assignments, feedback, and reports.
- Implement Oracle PL/SQL automation for SLA, escalation, audit trails, and performance scoring.
- Build role-based portals for Student, Worker, Supervisor, and Admin with a shared Node.js API.
- Provide operational dashboards for queues, overdue work, chronic locations, and monthly analytics.
- Demonstrate end-to-end workflow with seed data, authentication, and a public GitHub repository.

3. Problem Statement & Motivation

Without a centralized system, maintenance requests are lost in informal channels. Supervisors cannot see workload, SLA risk, or recurring location failures. Workers lack a clear assignment queue. Students receive little feedback after reporting an issue. A database-centric solution is ideal because integrity, history, and analytics all depend on durable relational data and server-side constraints rather than UI-only validation.

4. System Overview & Key Features

4.1 Core Features

- Complaint submission with category, priority, location, and description.
- Supervisor assignment of workers with specialization awareness.
- Worker job lifecycle: start, resolve, log repair cost, toggle availability.
- Student feedback/rating after resolution.
- Monthly maintenance report generation with totals, averages, and cost.

4.2 Enhanced Database Features

- **SLA tracking:** urgent = 4 hours, medium = 24 hours, low = 72 hours (trigger).
- **Auto-escalation:** overdue assigned complaints can be escalated for supervisor attention.
- **Audit log:** every status transition is recorded in STATUS_LOG.
- **Chronic detection:** 3+ same category at the same location creates a chronic flag.
- **Performance score:** worker score derived from resolution time and feedback ratings.
- **Budget tracking:** repair cost stored per assignment and rolled into monthly reports.

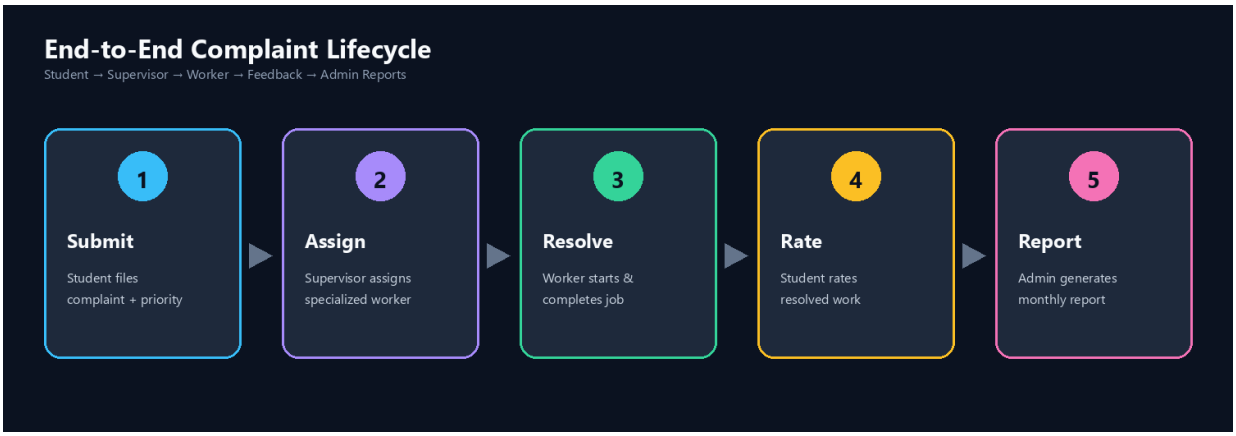

Figure 1. End-to-end complaint lifecycle across system roles.

5. Technology Stack

Layer	Technology
Database	Oracle Database 21c XE — SQL, PL/SQL, sequences, triggers, procedures, functions
Backend API	Node.js, Express, oracledb, session/token auth, REST endpoints
Frontend	HTML5, modern CSS, vanilla JavaScript (role portals)
Tooling	DBeaver / SQL Developer, GitHub, dbdiagram.io (ERD source)

6. Role-Based Workflow

The command chain is: **Student submits** → **Supervisor assigns** → **Worker resolves** → **Student rates** → **Admin reports**. Supervisors and Admins share the operations dashboard; Admins additionally receive the Directory tab for account and location management.

Role	Portal	Responsibilities
Student	/student.html	Submit, track SLA, search/filter, rate resolved work
Worker	/worker.html	Active/done jobs, SLA urgency, start/resolve, availability
Supervisor	/admin.html	Assign, queue, overdue, workers, chronic, reports
Admin	/admin.html	Full ops + Directory (students/workers/locations/CSV import)

Figure 2. Role-based access model.

7. System Architecture

The system follows a classic three-tier architecture. The browser hosts role-specific portals. Express exposes authenticated REST APIs. Oracle stores authoritative state and executes business rules close to the data for consistency and auditability.

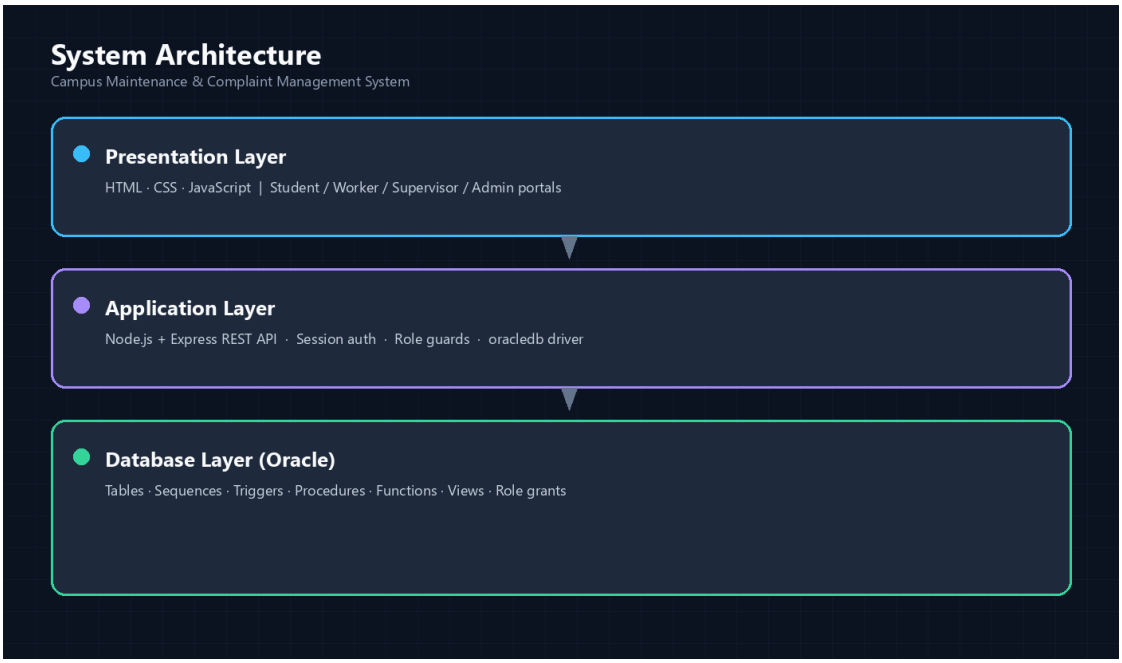

Figure 3. Three-tier system architecture.

Project layout groups DDL/DML/PLSQL under **sql/**, the API and static UI under **frontend/**, design documents under **docs/**, and screenshots under **assets/**.

8. Database Design

8.1 Entity Relationship Diagram

The schema contains nine core tables. COMPLAINTS is the central fact entity linking students and locations. ASSIGNMENTS connect workers to complaints. STATUS_LOG and FEEDBACK preserve history and quality signals. CHRONIC_FLAGS and MAINTENANCE_REPORTS support operations analytics.

Figure 4. Conceptual ER overview generated for this report.

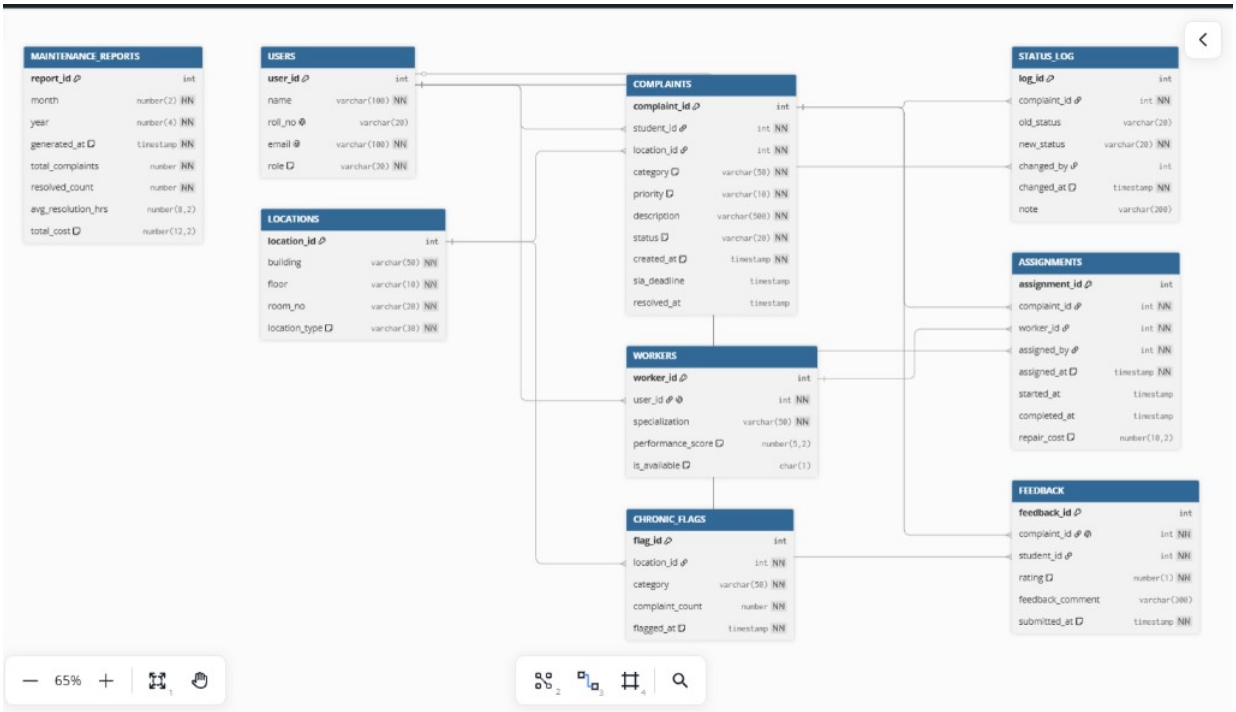

Figure 5. Relational schema diagram (table relationships view).

8.2 Tables Overview

Table	Purpose	PK
USERS	All actors (student/worker/supervisor/admin)	user_id
LOCATIONS	Campus building / floor / room registry	location_id
COMPLAINTS	Maintenance requests with SLA & status	complaint_id

Table	Purpose	PK
WORKERS	Worker profile + specialization + score	worker_id
ASSIGNMENTS	Worker–complaint link, timing, repair cost	assignment_id
STATUS_LOG	Immutable audit trail of status changes	log_id
FEEDBACK	Student rating after resolution	feedback_id
CHRONIC_FLAGS	Recurring location + category alerts	flag_id
MAINTENANCE_REPORTS	Monthly aggregated operations snapshot	report_id

8.3 Normalization (3NF)

- **1NF:** atomic attributes, unique rows, no repeating groups.
- **2NF:** surrogate single-column keys; no partial dependencies.
- **3NF:** worker-specific fields isolated in WORKERS; history in STATUS_LOG; ratings in FEEDBACK.
- **Controlled denormalization:** cached sla_deadline and performance_score for query speed.

9. PL/SQL Business Logic

Database intelligence is a primary learning outcome of this project. Business rules are not left solely to the frontend; they are encoded as Oracle objects so every client path remains consistent.

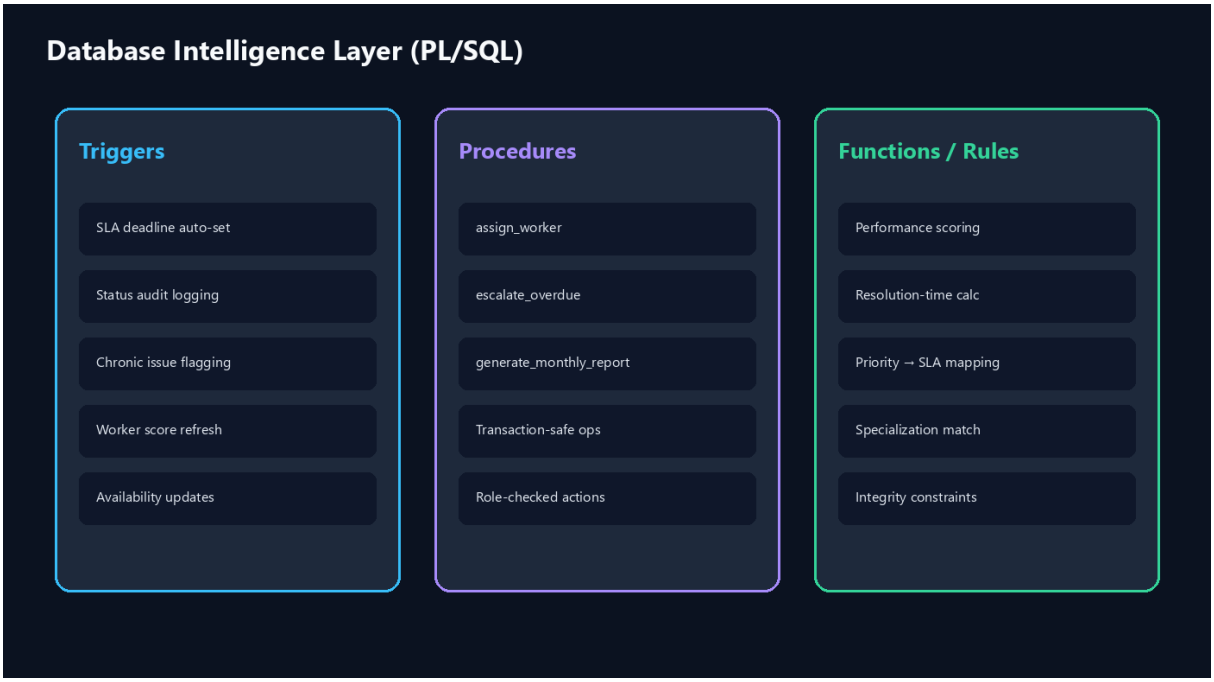

Figure 6. PL/SQL triggers, procedures, and functions summary.

9.1 Representative Rules

Rule	Enforcement
Priority → SLA deadline	Trigger on COMPLAINTS INSERT
Status change audit	Trigger writes STATUS_LOG rows
Chronic issue (≥3 same location+category)	Compound trigger on COMPLAINTS
Worker performance score refresh	Trigger/function after FEEDBACK
Assign specialized worker	Stored procedure assign_worker
Escalate overdue assigned work	Procedure escalate_overdue_complaints
Generate monthly report	Procedure writing MAINTENANCE_REPORTS

10. Implementation & User Interfaces

The frontend is a responsive dark-themed operations UI with shared navigation patterns, role guards, and live dashboard metrics. Below are representative screenshots from the running system.

10.1 Authentication

Users sign in with email and password. Demo accounts use the shared initial password **Password123**. A forgot-password flow issues a short-lived reset code (shown in the UI when DEMO_MODE is enabled).

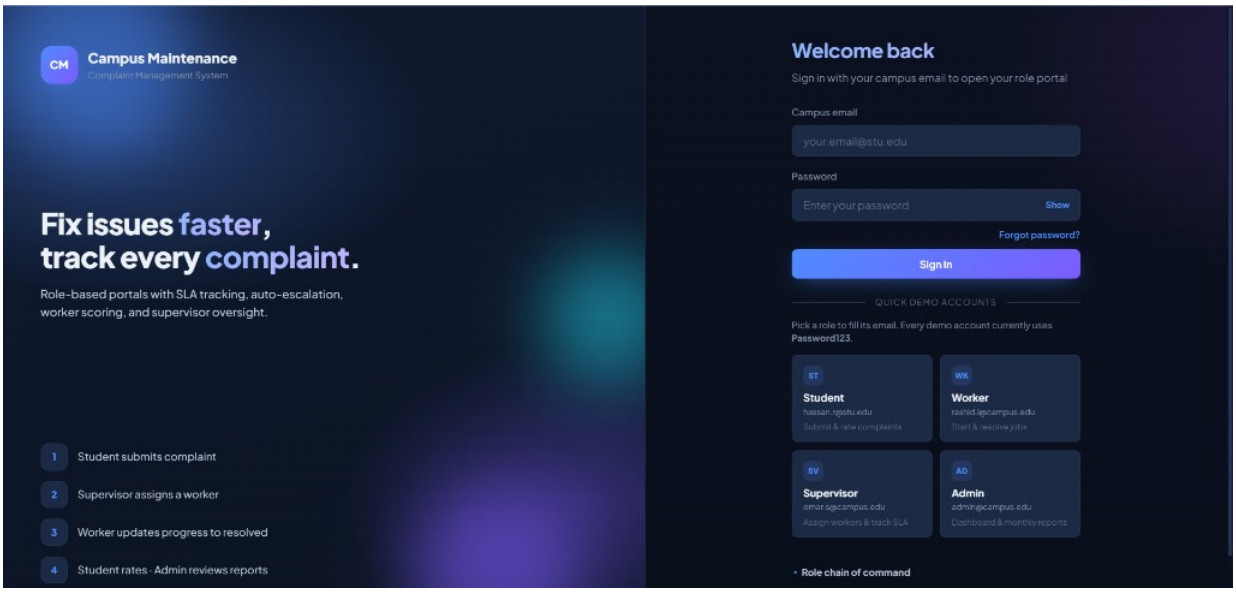

Figure 7. Login portal with role-aware access.

10.2 Student Portal

Students submit complaints, filter/search their history, monitor SLA remaining time, and rate resolved work through a modal feedback flow.

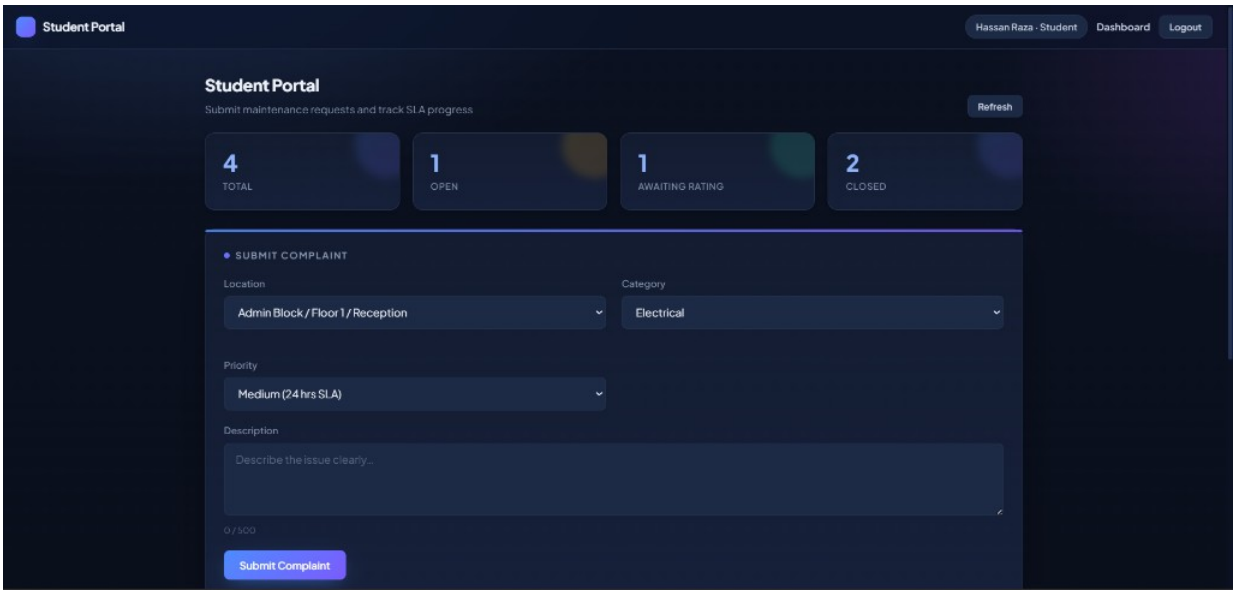

Figure 8. Student portal overview.

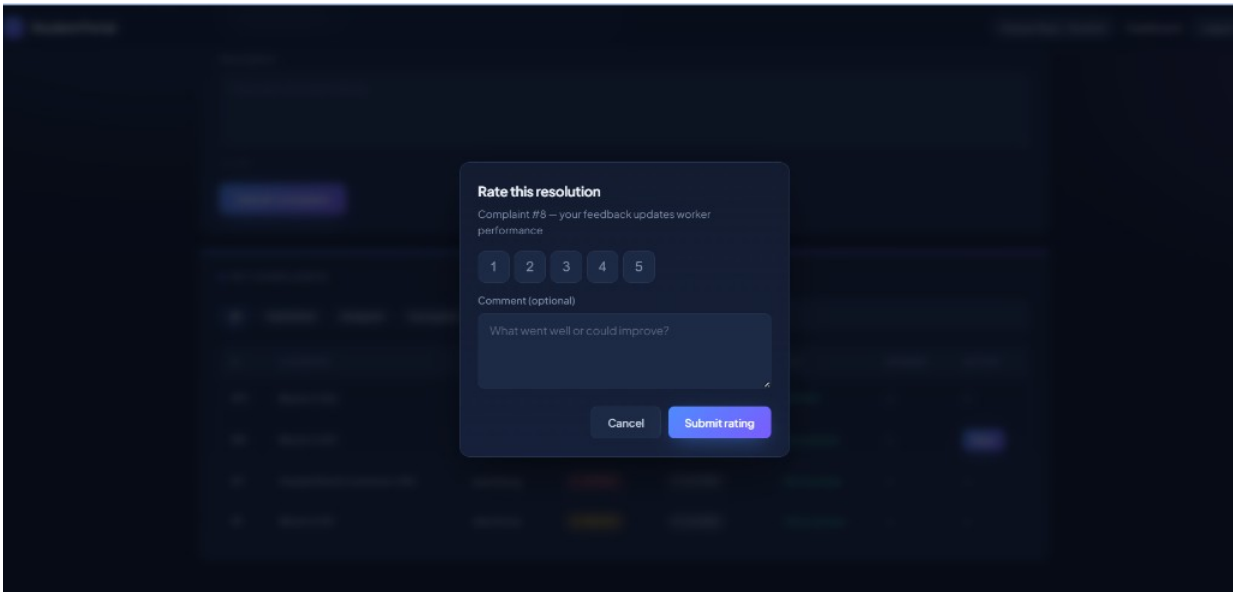

Figure 9. Student complaint tracking / details.

10.3 Worker Portal

Workers view assigned jobs, separate active vs completed work, start and resolve tasks, and toggle availability for new assignments.

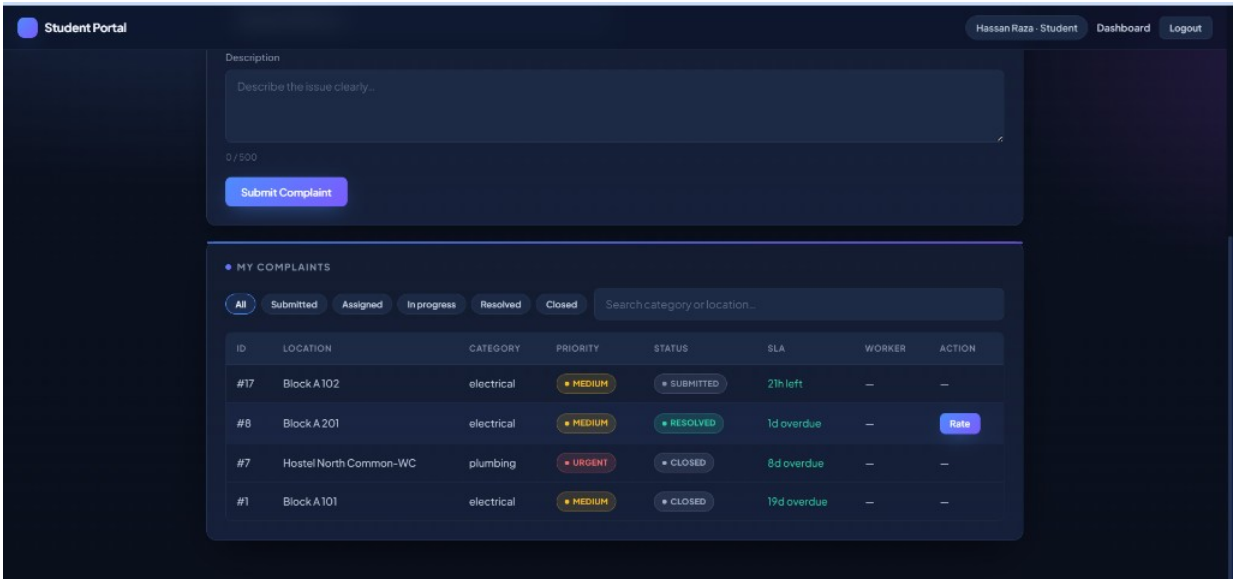

Figure 10. Worker portal.

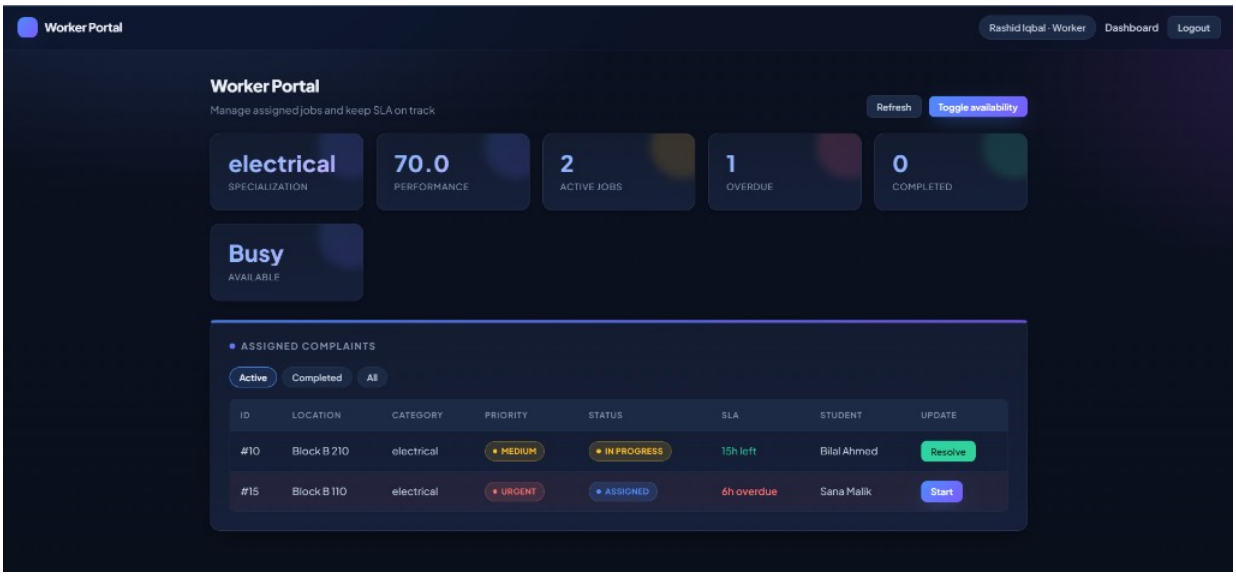

Figure 11. Worker job queue and actions.

10.4 Supervisor Operations Dashboard

Supervisors manage day-to-day operations: assign workers from the submitted queue, inspect overdue SLA breaches, review worker performance, investigate chronic locations, and open reports.

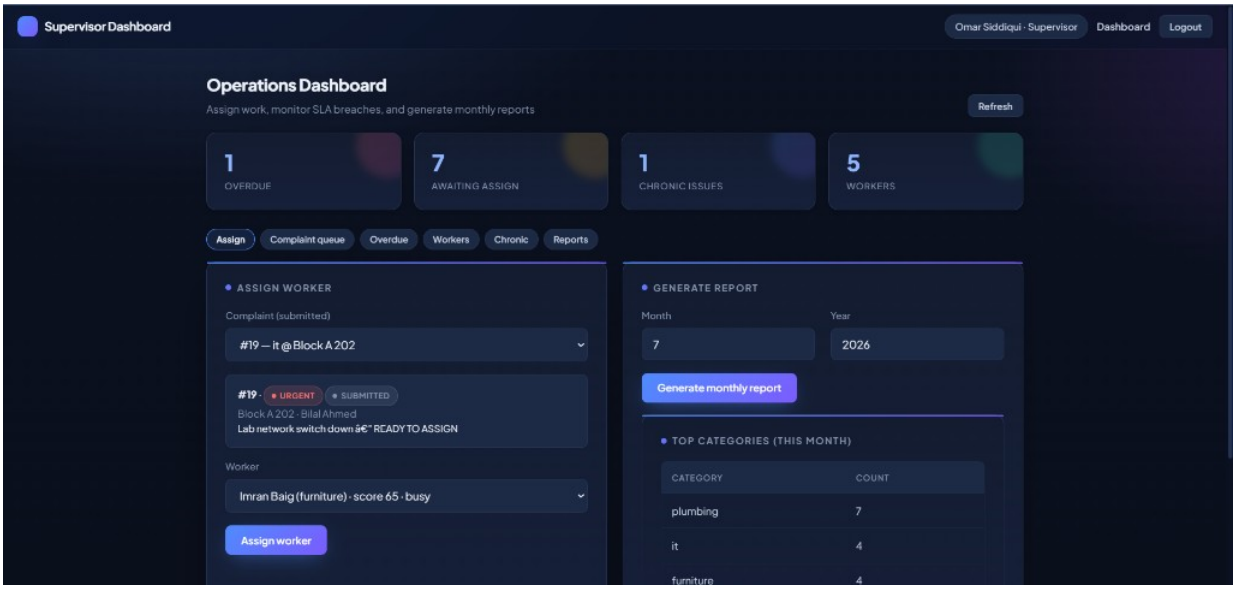

Figure 12. Supervisor — Assign worker.

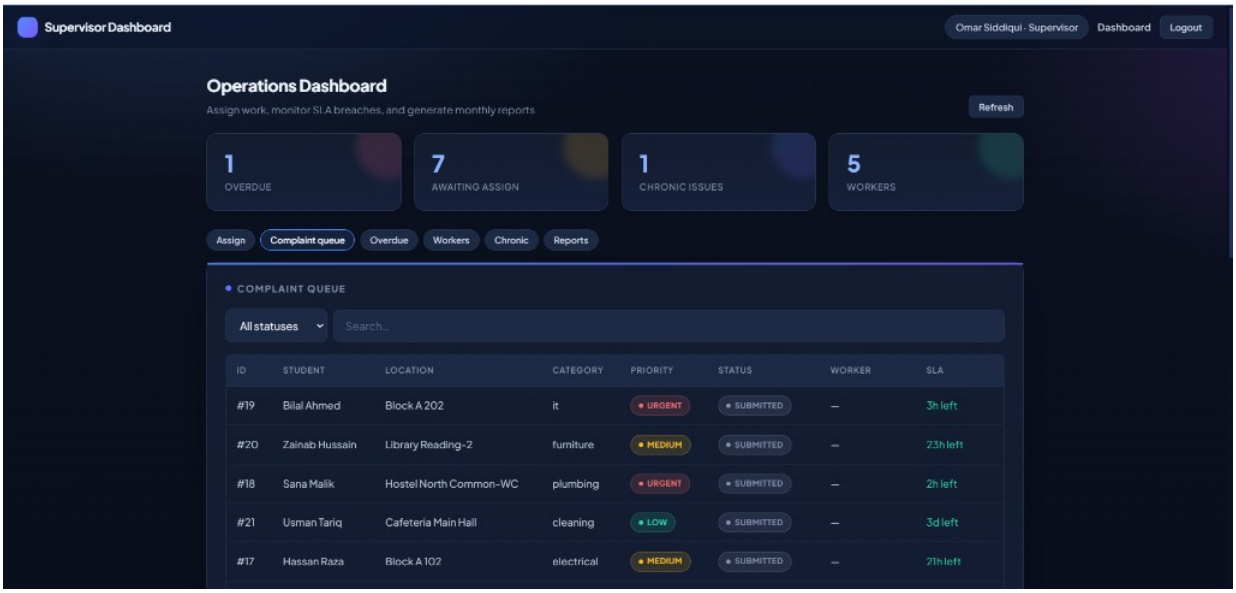

Figure 13. Supervisor — Complaint queue.

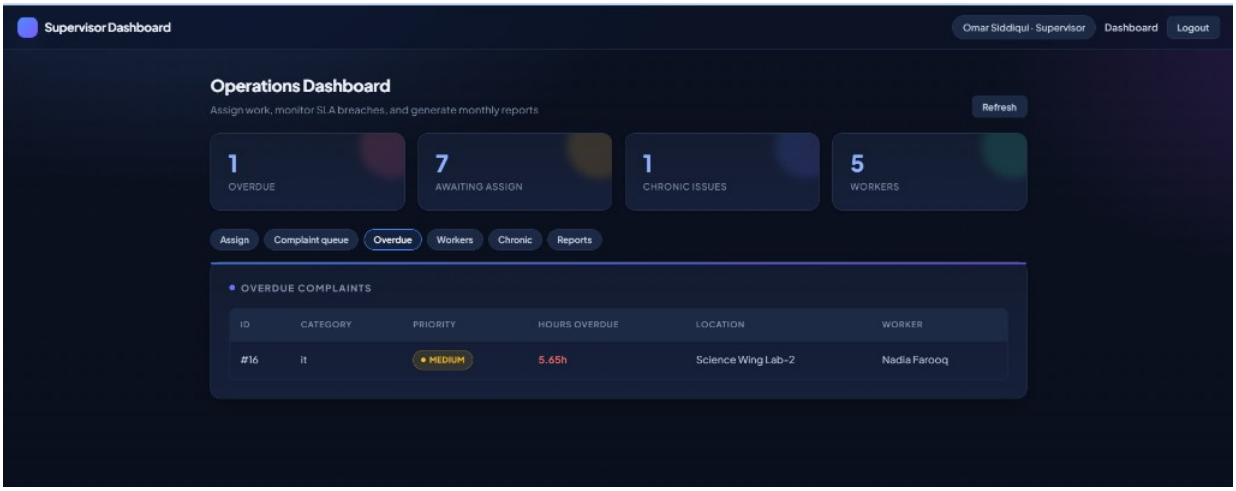

Figure 14. Supervisor — Overdue complaints.

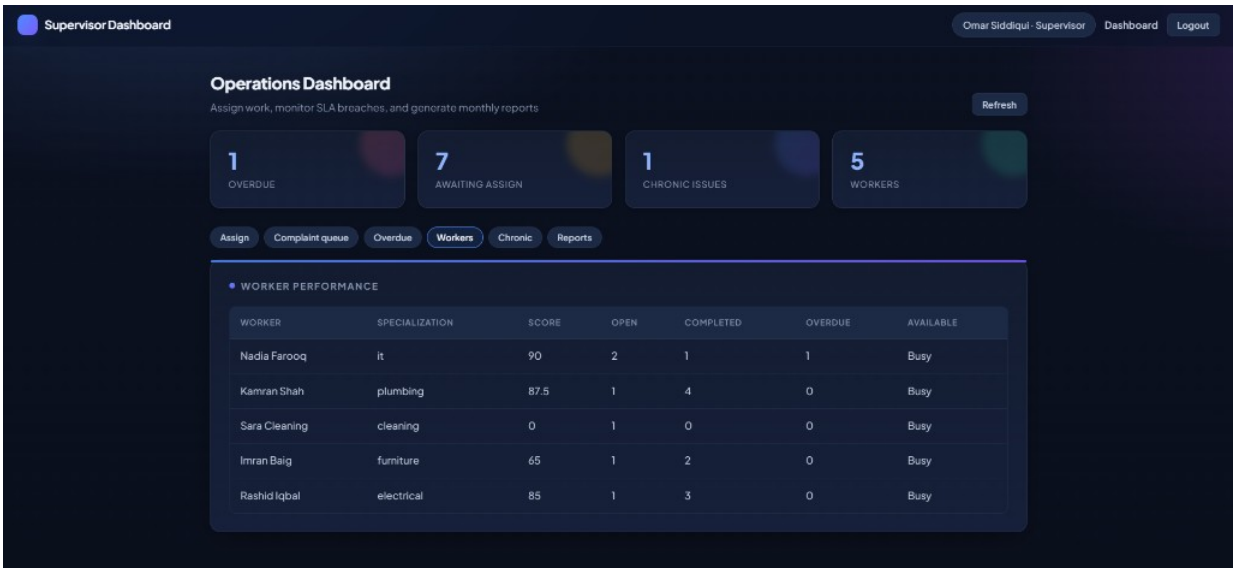

Figure 15. Supervisor — Worker performance.

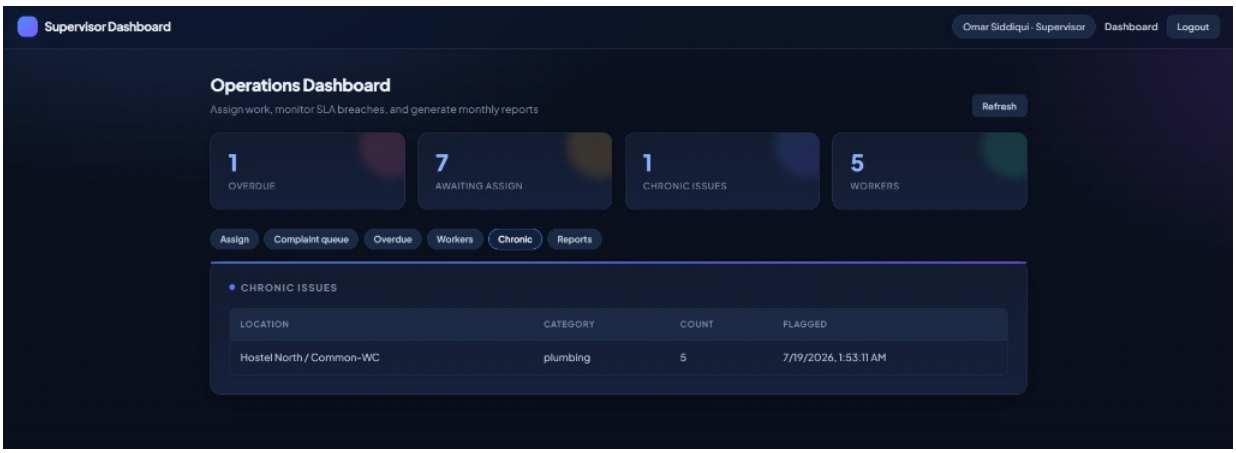

Figure 16. Supervisor — Chronic issues.

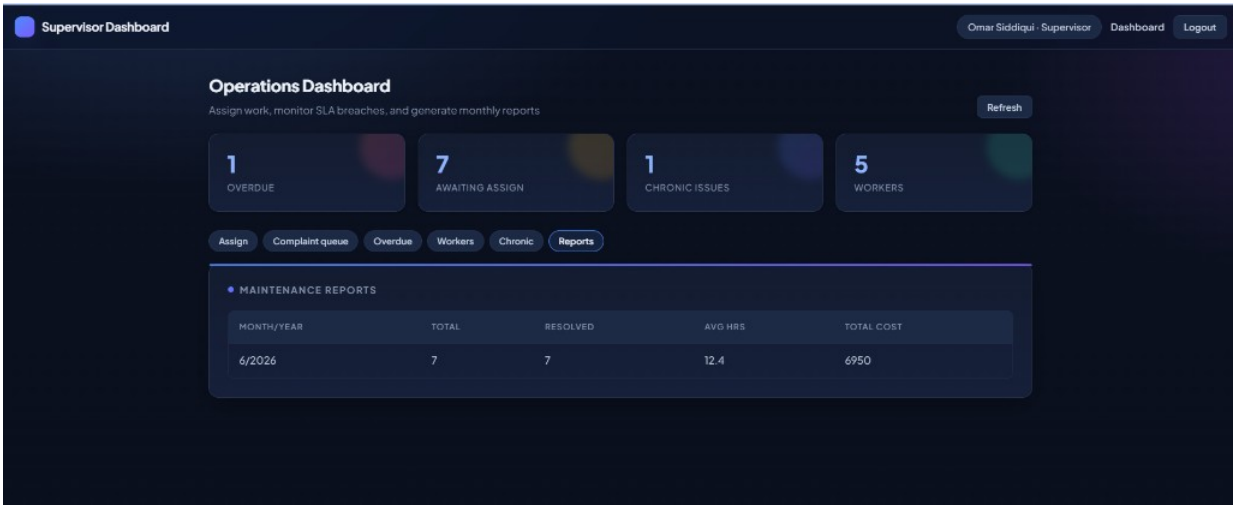

Figure 17. Supervisor — Maintenance reports.

10.5 Admin Dashboard & Directory

Admins inherit the operations dashboard and gain Directory management: add students (including CSV bulk import), add workers with specialization, register locations, and browse directories.

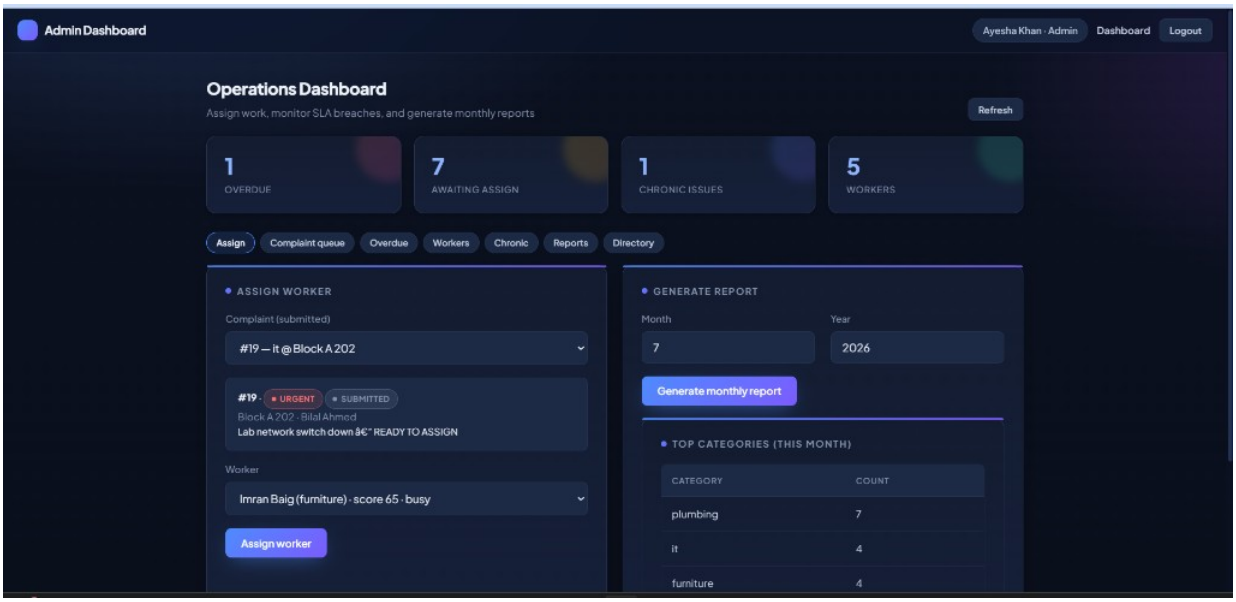

Figure 18. Admin — Operations assign view.

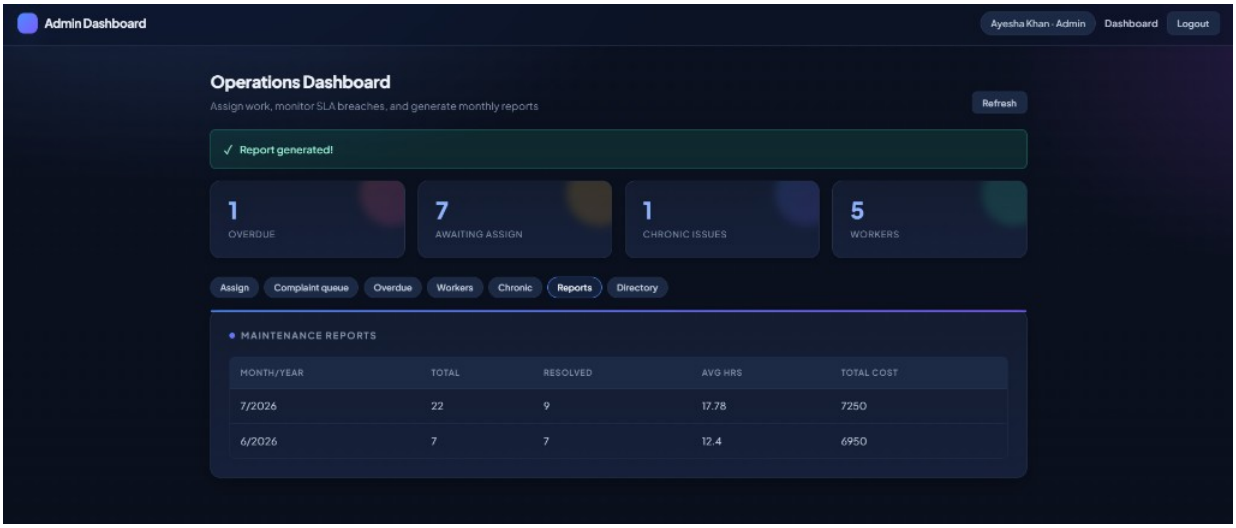

Figure 19. Admin — Generated monthly reports.

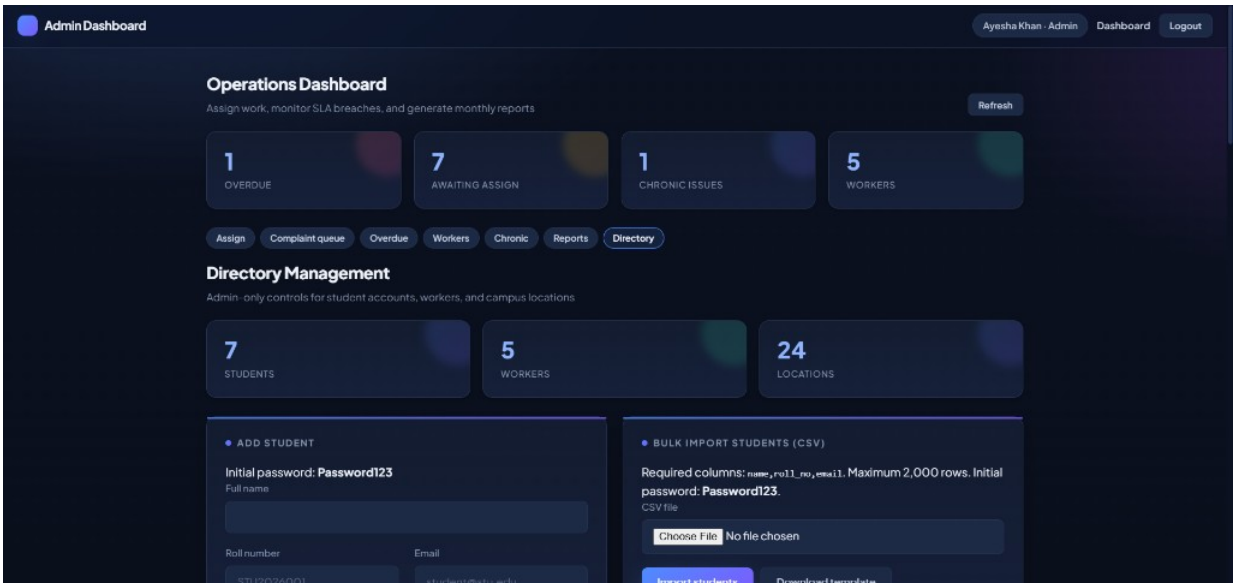

Figure 20. Admin — Directory overview.

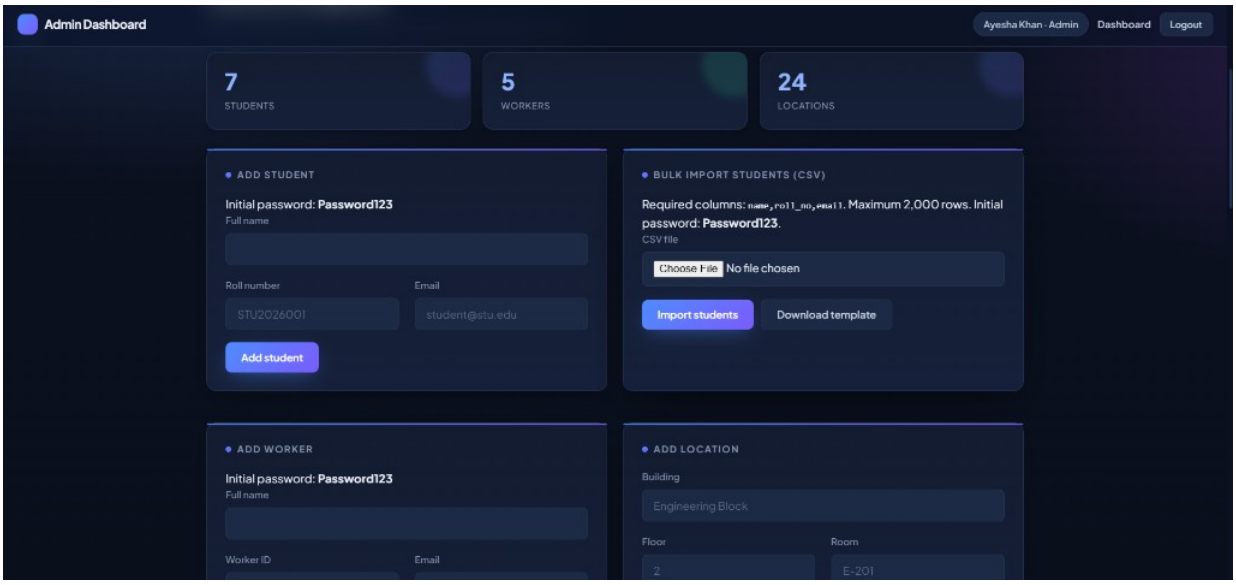

Figure 21. Admin — Add / import students.

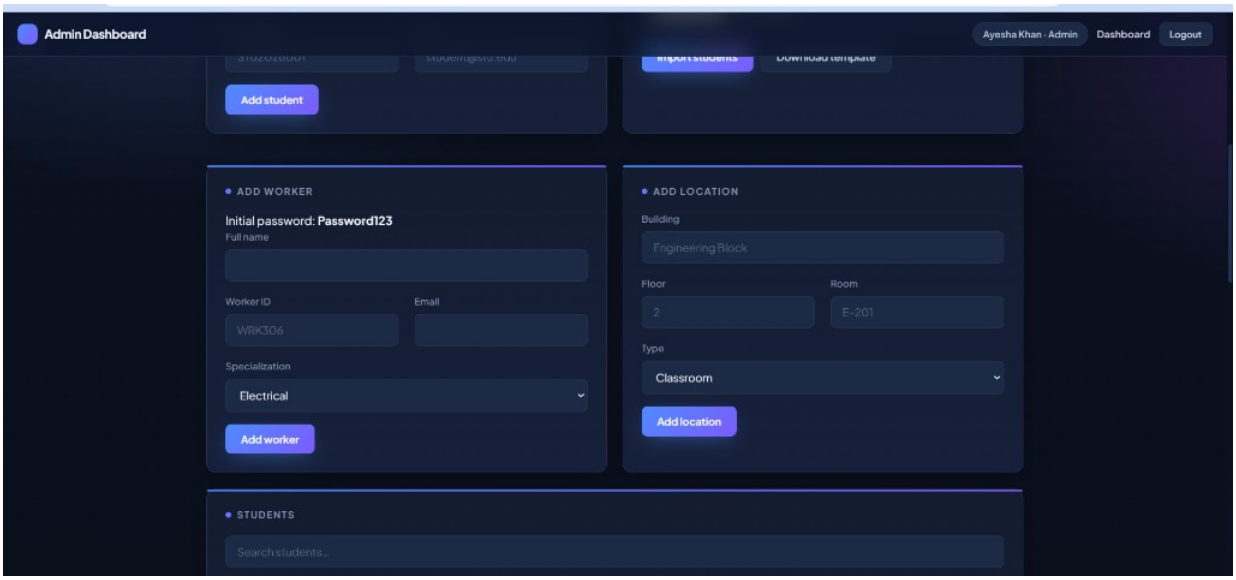

Figure 22. Admin — Add worker / location forms.

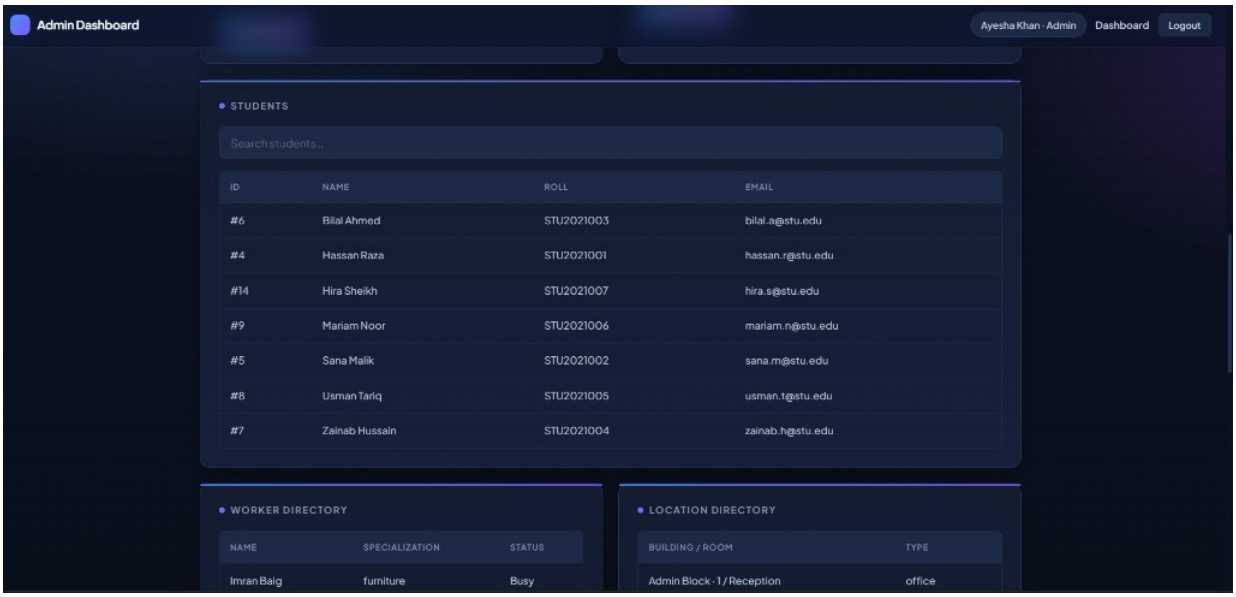

Figure 23. Admin — Students & directories.

11. Security & Authentication

- Password hashes stored in the database (scrypt-based hashing in the Node API).
- Bearer/session token required for privileged Admin Directory APIs.
- Role guards prevent students/workers from accessing supervisor/admin actions.
- Supervisors cannot use Admin-only Directory mutation endpoints (403).
- Password reset tokens are time-limited; demo mode surfaces codes for viva convenience.
- Oracle CHECK constraints and FK relationships protect data integrity independently of the UI.

12. Testing & Demo Accounts

Seed scripts and a demo-reset script populate a realistic campus dataset (students, workers, locations, mixed complaint statuses, overdue work, and at least one chronic plumbing location). Default password for demo accounts: **Password123**.

Role	Email	Password
Student	hassan.r@stu.edu	Password123
Worker	rashid.i@campus.edu	Password123
Supervisor	omar.s@campus.edu	Password123
Admin	admin@campus.edu	Password123

Local run: configure **frontend/.env**, start Oracle listener/service, run SQL scripts, then **npm start** in **frontend/** and open http://localhost:3000.

13. Repository & Development History

Source code, SQL scripts, documentation, and screenshots are hosted at https://github.com/ATSadi/dB-proj/.

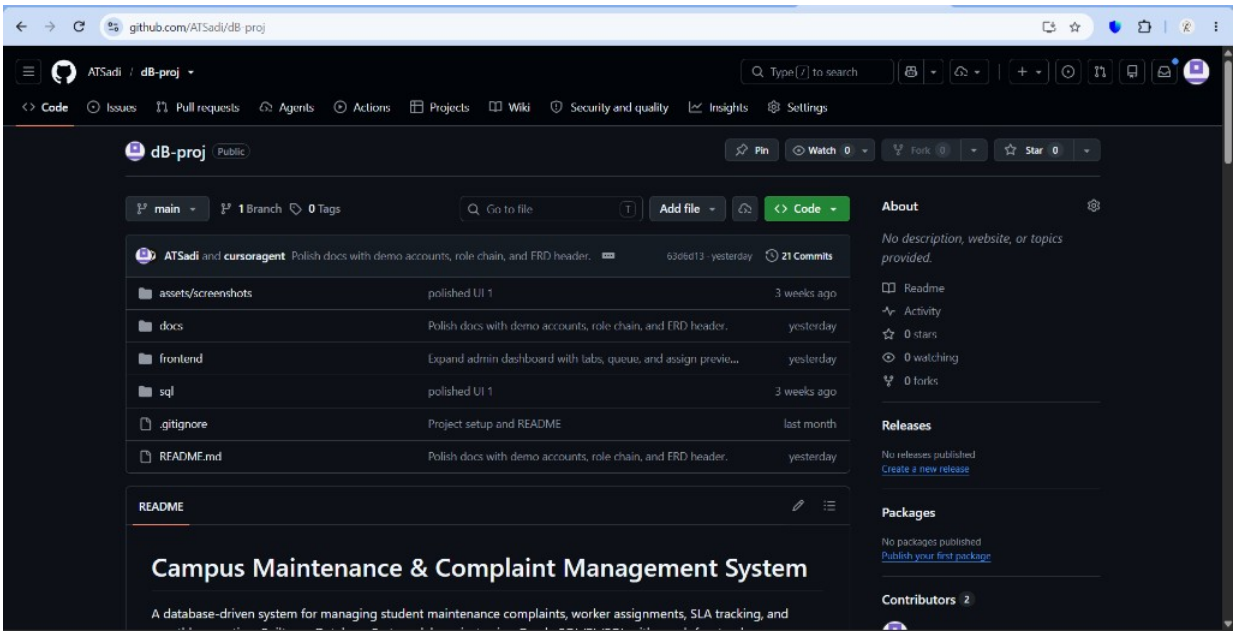

Figure 24. GitHub repository landing page.

Figure 25. Commit history highlighting database → API → UI progression.

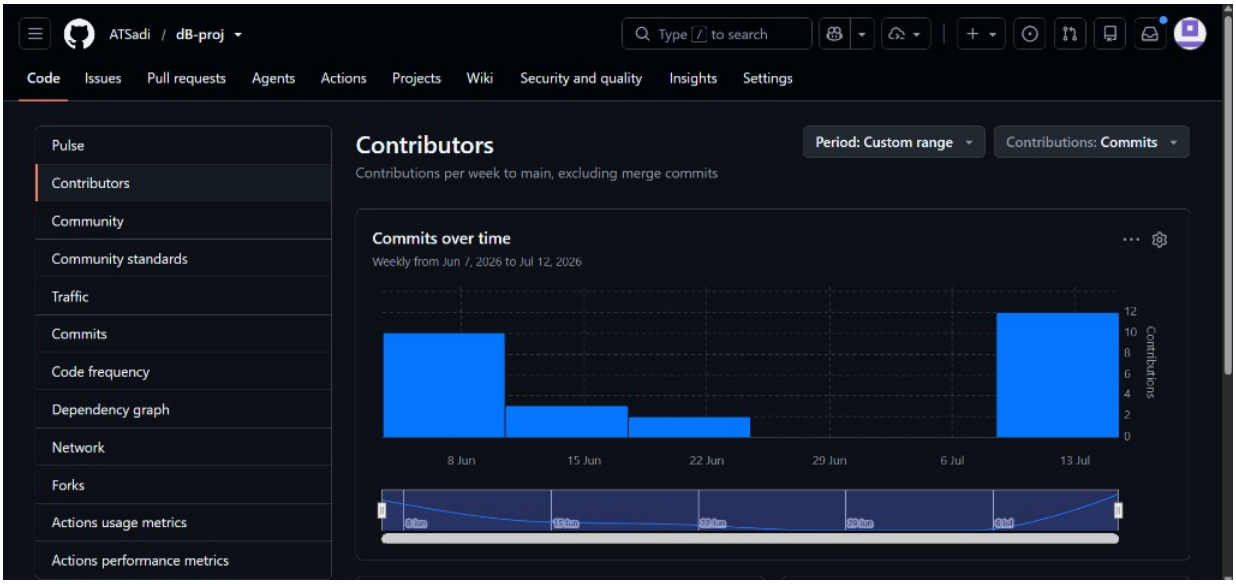

Figure 26. Contributors / commit activity insight.

Development progressed from schema/ERD foundations and PL/SQL automation, through Node.js API integration and analytical queries, to polished role portals (student filters, worker availability, admin/supervisor dashboards) and documentation for viva demos.

14. Conclusion & Future Work

The project successfully demonstrates a production-shaped academic system: normalized schema design, database-enforced business rules, multi-role workflows, operational analytics, and a usable web interface. It shows how Oracle PL/SQL and a lightweight Node/Express layer can cooperate without sacrificing integrity.

Future extensions may include:

- Email/SMS notifications for SLA risk and assignment events.
- Photo attachments for complaint evidence (object storage).
- Advanced analytics dashboards (heatmaps by building/category).
- Mobile-first PWA packaging for on-site workers.
- Deeper Oracle role grants mapped 1:1 with application permissions.

15. References

- Oracle Database Documentation — SQL Language Reference & PL/SQL Language Reference.
- Elmasri, R. & Navathe, S. — Fundamentals of Database Systems (normalization & ER modeling).
- Express / Node.js documentation — https://expressjs.com/
- Project repository — https://github.com/ATSadi/dB-proj/
- Khulna University of Engineering & Technology — https://www.kuet.ac.bd/

— End of Report —